

Cos'è Docker e quale problema risolve?

Docker serve a eseguire un'applicazione in un ambiente isolato chiamato container (contiene tutto ciò che serve per far funzionare un'applicazione: codice, librerie, dipendenze e configurazioni). Risolve i problemi di differenze tra ambienti di sviluppo e produzione, garantendo che l'app funzioni allo stesso modo ovunque.

Differenza tra container e macchina virtuale?

I container sono più leggeri perché condividono il kernel del sistema operativo consumando meno risorse, mentre le macchine virtuali includono un intero sistema operativo con applicazione, librerie, risultando più pesanti.

Cos'è un Dockerfile e a cosa serve?

Il Dockerfile è un file che definisce le istruzioni per costruire un'immagine Docker ci sono comandi come :

FROM definisce l'immagine di base su cui costruisco il container

WORKDIR definisce la cartella in cui verranno eseguiti i comandi successivi.

COPY copia i file dal tuo progetto dentro il container

RUN esegue comandi durante la creazione dell'immagine, ad esempio per installare dipendenze.

CMD definisce il comando di default eseguito all'avvio del container.

Cos'è Docker Compose e perché si usa?

Docker Compose serve a definire e avviare più container insieme tramite un file docker-compose.yaml serve quando un'applicazione è composta da più servizi come backend ,frontend...

Architettura Docker ?

Docker segue un'architettura client-server composta da tre elementi principali:

1- docker client : è una interfaccia con cui l'utente interagisce con docker

2- docker daemon : è il motore di docker che costruisce le immagini , esegue il container , gestisce la rete

3- docker registry : è il luogo dove sono salvate le immagini docker come docker hub

Differenza tra POST e PUT?

POST serve per creare risorse ed è non idempotente, mentre PUT serve per aggiornare risorse ed è idempotente.

Idempotente : Una richiesta è idempotente se farla una o più volte produce **sempre lo stesso risultato**.

Safe : Un metodo è safe se non modifica lo stato del server.

Cacheable : Un metodo è cacheable se la risposta può essere **memorizzata e riutilizzata**.

Metodo	Cosa fa	Idempotent e	Safe	Cacheabl e
GET	Recupera dati	✓ Sì	✓ Sì	✓ Sì

POST	Crea una risorsa	✗ No	✗ No	✗ No
PUT	Aggiorna/sostituisce una risorsa	✓ Sì	✗ No	✗ No
PATCH	Aggiorna parzialmente	✗ No	✗ No	✗ No
DELETE	Elimina una risorsa	✓ Sì	✗ No	✗ No

Cos'è un'API RESTful e quali sono i suoi principi?

REST (Representational State Transfer) è uno stile architetturale per sistemi distribuiti, che si basa sul trasferimento della rappresentazione delle risorse tra client e server tramite HTTP. Un sistema RESTful deve rispettare alcuni vincoli fondamentali: **client-server** (separazione tra interfaccia e logica), **stateless** (ogni richiesta contiene tutte le informazioni necessarie), **cacheable** (le risposte possono essere memorizzate), **uniform interface** (uso metodi HTTP per manipolare risorse) e **layered system** (architettura a livelli in cui i componenti non conoscono quelli oltre il livello immediatamente adiacente).

HTTP Response Status Codes (codici di stato)?

Categoria	Codice	Nome	Significato
2xx	200	OK	Richiesta completata con successo
	201	Created	Risorsa creata (POST)
	204	No Content	Successo senza corpo di risposta
4xx	400	Bad Request	Richiesta non valida (errore client)
	401	Unauthorized	Utente non autenticato
	403	Forbidden	Utente senza permessi
	404	Not Found	Risorsa non trovata
5xx	500	Internal Server Error	Errore generico del server
	502	Bad Gateway	Errore tra server
	503	Service Unavailable	Server non disponibile

Differenza tra SOP e CORS?

La **SOP** (Same-Origin Policy) è la regola di sicurezza del browser che **blocca** le richieste tra origini (protocollo + dominio + porta) diverse; il **CORS** (Cross-Origin Resource Sharing) è il meccanismo che permette al server di dire al browser **quali origini esterne sono autorizzate**.

OpenAPI: cos'è e a cosa serve?

OpenAPI è uno standard che descrive in un file (YAML) come funziona una API REST: quali endpoint esistono, quali metodi HTTP usano, quali dati ricevono e quali risposte restituiscono. In pratica è un “contratto” tra frontend e backend, perché entrambi seguono la stessa definizione: prima si scrive la specifica OpenAPI, poi si implementa il backend in base a quella specifica e infine il frontend usa le stesse regole , così si evitano errori e tutto resta coerente.

Cos'è una Promise in JavaScript?

Una Promise è un oggetto che rappresenta il risultato futuro di un'operazione asincrona. Può essere pending, fulfilled o rejected e si gestisce tramite async/await.

Come funziona l'asincronia in JavaScript?

JavaScript è single-threaded, quindi esegue una cosa alla volta, ma usa l'asincronia per non bloccarsi. Le operazioni lente vengono eseguite fuori dal thread principale e il risultato viene gestito dopo, tramite callback, Promise o async/await oppure then/catch.

Come si gestiscono gli errori?

Gli errori si gestiscono per evitare blocchi del programma. In JavaScript si usa try/catch o .catch() con le Promise. In Go invece si usa il valore di errore restituito dalle funzioni, mentre panic (termina immediatamente il programma) si usa solo per errori gravi.

Differenza tra errore gestito e panic (in Go)?

In Go gli errori normali vengono gestiti tramite il valore error e permettono al programma di continuare. Il panic invece rappresenta un errore grave non recuperabile che interrompe l'esecuzione del programma.

Cosa sono le Promise e come si collegano all'asincronia?

Le Promise sono oggetti che rappresentano il risultato futuro di un'operazione asincrona. Servono a gestire l'asincronia in modo più chiaro e sono alla base di async/await.

Come comunicano i servizi tra loro?

I servizi in Docker Compose comunicano tramite una rete interna creata automaticamente. Ogni servizio è raggiungibile tramite il proprio nome, che funge da hostname DNS. Ad esempio il frontend può chiamare il backend usando 'api-server:3000' invece di localhost.

Come costruisce docker l'immagine?

Docker costruisce le immagini come una serie di layer immutabili sovrapposti. Ogni istruzione del Dockerfile crea un nuovo layer che rappresenta una modifica. Quando si avvia un container, Docker unisce questi layer e aggiunge un layer scrivibile sopra, mentre quelli sottostanti restano in sola lettura.

Cos'è il DOM?

Il **DOM (Document Object Model)** è la rappresentazione della pagina HTML come **struttura ad albero** che JavaScript può modificare.

Come viene modificato il DOM in Vue.js?

In Vue.js il DOM non viene modificato direttamente. Si modificano i dati del componente e grazie al sistema di reattività Vue rileva i cambiamenti e aggiorna automaticamente il DOM.